

Decisões e trade-offs

Como defender a arquitetura da Kanban API em uma entrevista
Back-End Sênior

Método: problema → alternativas → decisão → motivo → custo → quando evoluir.

Meta: compreender o raciocínio; evitar respostas decoradas.

Projeto: github.com/AlvaroMGueiros/kanban-api

1. Como um sênior explica decisões

Uma resposta madura não começa pelo nome de uma tecnologia. Ela começa pelo problema. Use sempre esta estrutura:

1. Qual risco ou necessidade existia?
2. Quais alternativas eram viáveis?
3. Qual foi escolhida?
4. Por que ela serve melhor ao contexto?
5. Qual custo ou limitação foi aceito?
6. Em qual cenário a decisão deveria mudar?

Frase-base: “Eu escolhi X porque o risco principal era Y. A alternativa Z reduziria outro custo, mas adicionaria complexidade que não se justificava neste escopo. O custo aceito foi W; eu revisaria a escolha se a condição K aparecesse.”

O que evitar

- “Usei porque é boa prática.”
- “É mais escalável” sem volume, métrica ou gargalo.
- “Clean Architecture manda fazer assim.”
- Defender toda decisão como definitiva.
- Esconder limitações do projeto.

2. Status derivado das datas

Problema	O status depende da data atual e pode mudar sem qualquer escrita.
Alternativas	Persistir status; atualizar por job; calcular em leitura.
Decisão	Calcular a partir do cronograma.
Motivo	Evita status desatualizado e elimina sincronização periódica.
Custo	Consultas por status ficam mais complexas.
Evolução	Em alto volume, avaliar projeção materializada, coluna gerenciada por processo ou modelo orientado a eventos.

“O status não é um dado independente; ele é uma projeção das datas. Persisti-lo criaria duas fontes de verdade.”

Pergunta de aprofundamento: e performance?

A derivação é barata por registro, mas o filtro exige predicados sobre datas. Os índices ajudam, porém consultas com OR podem demandar análise do plano. A resposta sênior é medir com volume real antes de materializar.

3. Política de domínio e Specification SQL

Problema	O domínio calcula status em Java, mas a listagem precisa filtrar antes da paginação.
Alternativas	Filtrar a página em memória; carregar tudo; traduzir a regra para SQL.
Decisão	Política Java para objetos e Specification para consulta.
Motivo	Totais e páginas permanecem corretos e o banco processa o conjunto.
Custo	Duas representações da mesma regra podem divergir.
Mitigação	Testes de integração com cenários equivalentes e precedência explícita.

“Não filtrei depois da paginação porque isso poderia entregar página vazia com `totalElements` incorreto. Aceitei a tradução SQL e protegi a equivalência com testes.”

4. Clock e tempo de negócio

Problema	Regras dependem de hoje e do fuso correto.
Alternativas	<code>LocalDate.now()</code> ; passar a data em todos os controllers; injetar <code>Clock</code> .
Decisão	<code>Clock</code> configurável, padrão America/Fortaleza.
Motivo	Testes determinísticos e uma referência temporal consistente por operação.
Custo	Dependência adicional nos services e configuração de fuso.
Evolução	Em sistema multi-região, fuso poderia pertencer à organização ou ao projeto.

“Capturo hoje uma vez por caso de uso. Assim, a operação não atravessa a meia-noite com dois valores diferentes.”

5. Limite do dia do vencimento

Problema	O enunciado exige término maior que hoje para EM_ANDAMENTO, mas não define o estado quando o término é exatamente hoje.
Alternativas	Rejeitar; considerar atrasado; manter em andamento durante o dia.
Decisão	Considerar em andamento até o final do dia.
Motivo	Atraso começa depois do prazo; um registro válido não deve passar a causar erro pela virada para o dia do vencimento.
Custo	Interpretação inclusiva de uma condição escrita como “maior que”.
Mitigação	ADR, README e testes de boundary.

“A especificação tem uma lacuna no limite. Escolhi estabilidade operacional e documentei a interpretação.”

6. Serviço específico de transições

Problema	Alterar status diretamente poderia contradizer as datas.
Alternativas	Setter; regras no controller; regras na entidade; serviço de domínio.
Decisão	<code>KanbanTransitionService</code> puro.
Motivo	Centraliza a tabela, aplica efeitos e valida o estado resultante.
Custo	Tabela explícita com vários ramos e mensagens.
Evolução	Se surgirem dezenas de estados, considerar máquina de estados ou tabela declarativa.

```
origem = calcular(cronograma)
novoCronograma = aplicarEfeitoPermitido(origem, destino)
resultado = calcular(novoCronograma)
se resultado != destino: rejeitar
senão: persistir
```

7. Atomicidade e cronograma por valor

Problema	Uma transição inválida não pode deixar datas parcialmente alteradas.
Decisão	Criar novo <code>ProjectSchedule</code> , validar e só depois substituir na entidade dentro da transação.
Motivo	Reduz estados intermediários observáveis e permite rollback.
Custo	Pequena alocação adicional de objeto.
Evidência	Teste de integração confirma 422 e cronograma inalterado.

8. CONCLUIDO para A_INICIAR

Problema	A tabela oficial não define ação automática; pede edição das datas.
Alternativas	Limpar datas automaticamente; bloquear e orientar.
Decisão	Bloquear sem mutação.
Motivo	Evita inferir alterações não autorizadas pelo requisito.
Custo	Cliente precisa editar antes de repetir a transição.

“Preferi uma ação explícita do cliente a apagar informação implicitamente.”

9. Many-to-many direto

Problema	Projetos têm vários responsáveis e vice-versa.
Alternativas	Many-to-many; entidade <code>ProjectResponsible</code> .
Decisão	Many-to-many unidirecional.
Motivo	O vínculo não possui atributos ou comportamento.
Custo	Evolução exige remodelagem se o vínculo ganhar dados.
Quando mudar	Função no projeto, horas, alocação, período, responsável principal ou histórico.

10. Secretaria como texto

Problema	Responsável precisa de secretaria, mas não existe catálogo oficial.
Alternativas	String; enum inventado; entidade com CRUD.
Decisão	String validada.
Motivo	Atende ao obrigatório e não inventa valores de negócio.
Custo	Pode haver variações semânticas no cadastro.
Quando mudar	Se opções precisarem ser controladas, criar entidade; não enum, pois órgãos podem mudar sem deploy.

“Um enum seria uma lista de negócio inventada e acoplada ao deploy. Uma entidade é a evolução correta quando existir catálogo oficial.”

11. Entidades de domínio anotadas com JPA

Problema	Isolar totalmente persistência poderia duplicar o modelo.
Alternativas	Modelo puro + modelo JPA + mapeadores; entidades anotadas.
Decisão	Entidades do domínio com JPA.
Motivo	Os modelos seriam praticamente iguais neste escopo.
Custo	Acoplamento às anotações de persistência.
Mitigação	Regras centrais continuam estáticas/puras e testáveis sem Spring.

12. DTOs e mapeamento manual

Problema	Entidades JPA não devem definir o contrato público.
Alternativas	Expor entidades; MapStruct; mapeadores manuais.
Decisão	Records e mapeadores manuais pequenos.
Motivo	Contrato explícito sem ferramenta adicional.
Custo	Código de mapeamento escrito à mão.
Quando mudar	Muitos modelos e transformações repetitivas poderiam justificar MapStruct.

13. Flyway versus geração automática

Problema	O schema precisa ser reproduzível e auditável.
Alternativas	<code>ddl-auto=create/update</code> ; scripts manuais; Flyway.
Decisão	Flyway cria; Hibernate valida.
Motivo	Mudanças explícitas, ordenadas e revisáveis.
Custo	Toda evolução exige migration.
Benefício	O mesmo histórico funciona localmente, em teste e em produção.

14. PostgreSQL com Testcontainers

Problema	H2 e mocks não validam SQL, schema e constraints reais.
Alternativas	Mocks; H2; banco compartilhado; Testcontainers.
Decisão	PostgreSQL descartável por suíte.
Motivo	Fidelidade e isolamento.
Custo	Docker obrigatório e suíte mais lenta.
Evidência	Migrations, precisão temporal, constraints, índices e transações são testados.

15. Constraints no domínio e no banco

Problema	Validação só na API pode ser contornada por outra integração ou concorrência.
Decisão	Bean Validation no contrato, invariantes no domínio e constraints no banco.
Custo	Algumas regras aparecem em mais de um nível.
Motivo	Cada nível protege uma fronteira diferente.

“Bean Validation melhora a resposta ao cliente; domínio protege o caso de uso; banco preserva integridade independentemente da origem da escrita.”

16. Códigos HTTP

Código	Semântica escolhida	Exemplo
400	Formato ou parâmetro inválido.	JSON malformatado, paginação inválida.
404	Recurso inexistente.	Projeto não encontrado.
409	Conflito com estado persistido.	E-mail único, responsável vinculado.
422	Entrada compreensível, inviável pelas regras.	Datas ou transição incompatível.

17. Índices e busca textual

- Decisão** Índices em vínculo, secretaria normalizada e datas usadas no status.
- Custo** Espaço e escrita mais cara.
- Não feito** B-tree para `%texto%`, pois não ajudaria a busca por substring.
- Quando mudar** Medir volume e `EXPLAIN ANALYZE`; se necessário, usar `pg_trgm` com índice GIN.

18. Indicador com quatro contagens

- Problema** Exibir quantidade por status mantendo a mesma semântica.
- Decisão** Reutilizar a Specification em quatro `count`.
- Motivo** Implementação pequena, legível e coerente.
- Custo** Quatro viagens ao banco.
- Quando mudar** Dashboard frequente ou grande volume justificaria consulta agregada, cache ou projeção.

19. Docker e CI

Docker	Build multi-stage reduz imagem final; runtime usa JRE e usuário não privilegiado.
Healthcheck	Compose conhece disponibilidade, não apenas processo ativo.
CI	Cada push em main e PR executa Java 21 + cache Maven + <code>mvn verify</code> .
Custo	Testcontainers consome mais tempo no runner.
Não feito	Deploy e imagem em registry, porque não eram necessários.

20. O que ficou fora e por quê

Item	Motivo	Evolução
GraphQL	Diferencial; REST cobria o núcleo.	Schema após estabilizar regras.
Autenticação	Fora do escopo obrigatório.	OIDC/JWT e autorização por perfil.
Prometheus	Health era suficiente para entrega.	Métricas de latência, erro e banco.
UI	Desafio de backend.	Kanban com drag-and-drop.
CRUD Secretaria	Diferencial sem catálogo fornecido.	Entidade, FK e desativação lógica.
Camada de IA	Diferencial; não deveria atrasar o núcleo.	ADR de RAG ou assistente.

21. Perguntas difíceis e respostas

“Você duplicou regra entre Java e SQL. Isso não viola DRY?”

DRY evita duplicação de conhecimento sem controle. Aqui existem duas representações necessárias em ambientes diferentes. A política Java é a referência; a tradução SQL existe por performance e paginação. Testes verificam equivalência. Se a regra crescesse muito, eu revisaria a estratégia.

“Por que não salvar status e indexar?”

Isso simplificaria consultas, mas introduziria consistência temporal. Eu só escolheria status persistido com uma estratégia explícita de atualização, tolerância a defasagem e necessidade comprovada por volume.

“O projeto está pronto para produção?”

O núcleo funcional está sólido, mas produção exigiria autenticação, segredos gerenciados, observabilidade, política de backup, deploy/rollback, limites de requisição, tratamento explícito de concorrência e testes de carga.

“Por que não criou interfaces para os services?”

Não havia segunda implementação nem fronteira que exigisse substituição. Interfaces sem necessidade aumentariam navegação e manutenção. Eu criaria quando existisse um contrato com múltiplos adaptadores ou isolamento real.

“Como sabe que os índices ajudam?”

Eles correspondem às colunas dos filtros, mas a confirmação definitiva depende de volume e plano de execução. Em produção eu usaria métricas e EXPLAIN ANALYZE; não trataria a simples existência do índice como prova.

22. Exercício para dominar o projeto

1. Explique status derivado sem usar a palavra “boa prática”.
2. Desenhe no papel o fluxo do PATCH de transição.
3. Explique por que 422 é diferente de 409.
4. Dê um caso em que Many-to-many deixaria de servir.
5. Defenda Clock e o fuso em 30 segundos.
6. Explique por que a Specification existe e qual risco ela cria.
7. Mostre um teste que prova rollback.
8. Liste três itens necessários antes de produção.
9. Explique uma sugestão de IA que foi rejeitada.
10. Apresente uma limitação sem desvalorizar a solução.

23. Fechamento para entrevista

“Minha prioridade foi tornar explícitas as invariantes temporais e impedir divergência entre intenção de status e cronograma. Mantive a arquitetura proporcional, testei o domínio isoladamente e validei persistência e HTTP com PostgreSQL real. Sei quais custos aceitei e em quais condições cada decisão precisaria evoluir.”

Use este guia junto do README, ADRs e testes. Se você consegue explicar cada decisão com suas próprias palavras, consegue defender o projeto sem depender de memorização.